EXPRESS.JS

Express.js

Express.js is a **minimal and flexible Node.js web framework** used for building web applications and APIs easily.

It simplifies:

- Routing
- Server creation
- Middleware handling
- Request & response management

Install Express:

npm install express

Express Setup

Steps to create an Express server.

Step 1: Install Express

npm install express

Step 2: Create Server

```
const express = require("express");

const app = express();

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Open browser:

http://localhost:3000

Nodemon Setup

Steps to install and use Nodemon (auto-restart server)

Step 1: Install Nodemon

Install globally:

npm install -g nodemon

Step 2: Run Server using Nodemon

Instead of:

node index.js

Use:

nodemon index.js

Step 3: Automatic Restart

- No need to restart manually
- Whenever you save file → server restarts automatically

Routing

Routing determines **how an application responds to client requests** for different URLs.

Example:

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Home Page");
});

app.get("/about", (req, res) => {
  res.send("About Page");
});

app.listen(3000);
```

Routes:

- / → Home Page
- /about → About Page

Different ways to write URL path

There are 3 main ways to write a URL path:

1. Static Path

- Fixed URL
- Does not change

```
app.get('/home', (req, res) => {
  res.send("Home Page");
});
```

Example: /home

---

2. Dynamic Path (Route Parameters)

- URL contains variables using :

```
app.get('/user/:id', (req, res) => {
  res.send(req.params.id);
});
```

Examples:

- /user/10
- /user/abc

---

3. Query Parameters

- Data is passed after ? in URL

```
app.get('/search', (req, res) => {
  res.send(req.query);
});
```

Example:
/search?name=reshma&age=20

Middleware

Middleware functions are functions that run between request and response.

They can:

- Modify request
- Modify response
- Execute code
- End request-response cycle

Example:

```
const express = require("express");
const app = express();
app.use((req, res, next) => {
  console.log("Middleware executed");
```

```
  next();
});

app.get("/", (req, res) => {
  res.send("Hello");
});

app.listen(3000);
```

Explanation:

- Middleware runs before route handler.

Express Middleware (Condition Check Example):

```
const express = require("express");

const app = express();

app.use((req, res, next) => {

  const isLoggedIn = false;

  if (isLoggedIn) {

    console.log("User authenticated");

    next(); // go to next middleware/route

  } else {

    res.send("Access Denied ");

  }

});

app.get("/", (req, res) => {

  res.send("Welcome to Home Page ");

});

app.listen(3000, () => {

  console.log("Server running on port 3000");

});
```

Request & Response

In Express.js:

- **Request (req)** → Information from client
- **Response (res)** → Data sent back to client

Common Request Properties:

1. req.params

- Used to get route parameters (dynamic values)

```
app.get('/user/:id', (req, res) => {
  res.send(req.params.id);
});
```

URL: /user/10
Output: 10

---

2. req.query

- Used to get query parameters (from URL)

```
app.get('/search', (req, res) => {
  res.send(req.query);
});
```

URL: /search?name=reshma&age=20
Output: { name: 'reshma', age: '20' }

---

3. req.body

- Used to get data sent from client (POST/PUT request)

```
app.use(express.json()); // important middleware

app.post('/data', (req, res) => {
  res.send(req.body);
});
```

Example Body:

```
{
  "name": "Reshma",
  "age": 20
}
```

Output:

{ name: 'Reshma', age: 20 }

---

4. req.headers

- Used to get request headers

```
app.get('/', (req, res) => {
  res.send(req.headers);
});
```

Example: content-type, authorization, etc.

---

5. req.method

- Shows HTTP method used

```
app.get('/', (req, res) => {
  res.send(req.method);
});
```

Output: GET

Common Response Methods:

1. res.send()

- Sends **any type of response** (text)

```
app.get('/', (req, res) => {
  res.send("Hello World");
});
```

---

2. res.json()

- Sends **JSON response**

```
app.get('/data', (req, res) => {
  res.json({ name: "Reshma", age: 20 });
});
```

---

3. res.status()

- Sets **HTTP status code**

```
app.get('/', (req, res) => {
  res.status(200).send("Success");
});
```

---

4. res.sendFile()

- Sends a **file as response**

```
res.sendFile(__dirname + '/index.html');
```

5. res.redirect()

- Redirects to another URL

res.redirect('/home');

HTTP Methods:

HTTP methods are used by the client to communicate with the server.

For now, we are NOT working with database or CRUD, only understanding how each method sends a response.

What is HTTP?

HTTP (HyperText Transfer Protocol) is a communication protocol between:

- Client (Browser / Postman)
- Server (Node.js / Express)

Common HTTP Methods

Method	Purpose	Example Use
GET	Get data	Fetch webpage
POST	Send data	Submit form
PUT	Update data	Edit details
DELETE	Remove data	Delete record

Express Example (Only Responses):

```
const express = require("express");
const app = express();

// GET Method
app.get("/", (req, res) => {
  res.send("This is GET request");
});

// POST Method
app.post("/", (req, res) => {
  res.send("This is POST request");
});

// PUT Method
app.put("/", (req, res) => {
```

```
  res.send("This is PUT request");
});

// DELETE Method
app.delete("/", (req, res) => {
  res.send("This is DELETE request");
});
```

Important Points:

- All methods can send response using res.send()
- Right now, we are not storing or modifying real data
- We are only learning how requests work

CRUD Operations

CRUD represents basic database operations.

Operation	Meaning
Create	Add data
Read	Get data
Update	Modify data
Delete	Remove data

Example:

```
const express = require("express");
const app = express();

let users = ["Reshma", "Anu"];

app.get("/users", (req, res) => {
  res.send(users);
});

app.post("/users", (req, res) => {
  users.push("New User");
  res.send("User Added");
});

app.put("/users", (req, res) => {
  users[0] = "Updated User";
  res.send("User Updated");
});
```

```
app.delete("/users", (req, res) => {
  users.pop();
  res.send("User Deleted");
});

app.listen(3000);
```

MVC Architecture

MVC stands for:

M → Model

V → View

C → Controller

It separates application logic into different parts.

Model

Handles database and data logic.

Example:
User schema, database queries.

View

Handles UI or presentation layer.

Example:
HTML pages or templates.

Controller

Handles **application logic and requests**.

Example:

```
app.get("/users", (req, res) => {
  res.send("User List");
});
```

Benefits of MVC:

- Better code organization
- Easy maintenance
- Reusable code

How MVC Works (Flow):

1. User sends request (Postman/Browser)

2. Controller receives request
3. Controller asks Model for data
4. Model interacts with database
5. Data goes back to Controller
6. Controller sends response to View

Microservices Architecture

Microservices architecture divides an application into **small independent services**.

Each service:

- Runs independently
- Handles a specific function
- Communicates using APIs

Example:

Instead of one big application:

E-commerce App

It can be divided into:

User Service
Product Service
Order Service
Payment Service

Advantages:

- Easy scaling
- Independent deployment
- Faster development
- Better fault isolation